

Parcial junio 2024

- I. Envidia de atributos(Línea 16 y 24)
- II. Aplico Move method
 - A. Defino método “calcularPrecio” en la clase “Producto” para realizar el cálculo del producto con la inclusión del IVA.
 - B. Muevo el código de cálculo de producto de las líneas 16 y 24 de la clase “Pago” al método “calcularPrecio” de la clase “Producto”.
 - C. Desde las líneas 16 y 24 de la clase “Pago” llamo al método “calcularPrecio” de la clase “Producto”.

III. `total = total + producto.calcularPrecio();`

- I. Código duplicado(Líneas 15 a 17 y en líneas 23 a 25)
- II. Aplico Extract Method
 - A. Creo un método llamado “calcularTotal”, copio el código de las líneas 15 a 17 y de las líneas 23 a 25 en el nuevo método.
 - B. Elimino el código repetido de las líneas 15 a 17 y 23 a 25.
 - C. En la variable “total” hago la llamada al método “calcularTotal”.

```

protected double calcularTotal()
{
    double total = 0;
    for(Producto producto : this.productos)
    {
        total = total + producto.calcularPrecio();
    }
    return total;
}

public double calcularMontoFinal(){
    double total = this.calcularTotal();
    if(this.tipo == "EFECTIVO")
    {
        if(total > 100000)
            total = total - DESCUENTO_EFECTIVO;
    }
    else if(this.tipo == "TARJETA")
    {
        total = total + ADICIONAL_TARJETA;
    }
    return total;
}

```

III.

- I. Bucle for (en el método “calcularTotal”)
- II. Aplico Replace Loop with Pipeline.
 - A. Reemplazo el bucle for del método “calcularTotal” por un stream.

```

protected double calcularTotal()
{
    double total = this.productos.stream().mapToDouble(p -> p.calcularPrecio()).sum();
    return total;
}

```

III.

- I. Temporary Field (en el método “calcularTotal”)
- II. Reemplazo la variable temporal “total” y todas sus llamadas en el método “calcularTotal” por la llamada al stream.

III.

```
protected double calcularTotal()
{
    return this.productos.stream().mapToDouble(p -> p.calcularPrecio()).sum();
}
```

Por el momento...

```
public class Pago {
    private List<Producto> productos;
    private String tipo;

    private static final double ADICIONAL_TARJETA = 1000.0;
    private static final double DESCUENTO_EFECTIVO = 2000.0;

    public Pago(String tipo, List<Producto> productos) {
        this.productos = productos;
        this.tipo = tipo;
    }

    protected double calcularTotal()
    {
        return this.productos.stream().mapToDouble(p -> p.calcularPrecio()).sum();
    }

    public double calcularMontoFinal(){
        double total = this.calcularTotal();
        if(this.tipo == "EFECTIVO")
        {
            if(total > 100000)
                total = total - DESCUENTO_EFECTIVO;
        }
        else if(this.tipo == "TARJETA")
        {
            total = total + ADICIONAL_TARJETA;
        }
        return total;
    }
}
```

- I. Switch Statements (en el método “calcularMontoFinal”)
- II. Aplico Replace Conditional With Polymorphism.
 - A. Creo una interfaz llamada “TipoDePago”
 - B. Por cada tipo de pago creo una clase, en este caso son las clases “TipoDePagoEfectivo” y “TipoDePagoTarjeta”.
 - C. Creo un método en la clase “TipoDePago” llamado “calcularPago”, defino este método en las clases “TipoDePagoEfectivo” y “TipoDePagoTarjeta”.

- D. Copio cada parte condicional del método “calcularMontoFinal” de la clase “Pago” en cada clase de tipo de pago correspondiente.
- E. En la clase “Pago” cambio el tipo de la variable “tipo” de String a TipoDePago(interfaz)
- F. Borro el método “calcularMontoFinal” de la clase “Pago” y lo reemplazo por los métodos “calcularMontoFinalEfectivo” y “calcularMontoFinalTarjeta” para los respectivos tipos de pago.
- G. En los métodos “calcularMontoFinalEfectivo” y “calcularMontoFinalTarjeta” simplemente llamo al método “calcularPago” de la variable “tipo” con descuento o recargo correspondiente.

III. Mostrar código.